



NATIONAL TECHNICAL UNIVERSITY OF ATHENS
SCHOOL OF ELECTRICAL AND COMPUTER ENGINEERING
INTERDISCIPLINARY M.SC. IN DATA SCIENCE AND MACHINE
LEARNING

Large-Scale Data Management

Academic Year 2025–26

Instructors: Dimitrios Tzoumakos, Ioannis Konstantinou

Lab Coordinator: Nikolaos Chalkvantzis

May 7, 2026

Semester Project

Description

This semester project requires the analysis of (large-scale) datasets by applying processing techniques commonly used in data science projects. The tools to be used in the context of the project are [Apache Hadoop](#) (version ≥ 3.0) and [Apache Spark](#) (version ≥ 3.5). You are expected to use the resources made available to you in the specially configured environment that has been set up for you, which you can access by following the course laboratory guides. In brief, the goals of the project are:

- to familiarize students with the installation and administration of the distributed systems Apache Spark and Apache Hadoop, and to help them develop the corresponding skills;
- to use modern techniques through the Spark APIs for large-scale data analysis;
- to understand the capabilities and limitations of these tools in relation to the available resources and the selected configuration settings.

Data

This section presents the datasets that you are required to use for the semester project. These are publicly available, free datasets collected from different sources.

Dataset	S3 URI
Los Angeles Crime Data (2010-2019)	hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/LA_Crime_Data/LA_Crime_Data_2010_2019.csv
Los Angeles Crime Data (2020-2025)	hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/LA_Crime_Data/LA_Crime_Data_2020_2025.csv
Census Blocks	hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/LA_Census_Blocks_2020.geojson
Census Blocks Fields	hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/LA_Census_Blocks_2020_fields.csv
Median Household Income by Zip Code	hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/LA_income_2021.csv
LA Police Stations	hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/LA_Police_Stations.csv

Table 1: Datasets and the locations where they are stored in HDFS.

For your convenience, all required datasets are accessible in the following HDFS directory:
[hdfs://hdfs-namenode.default.svc.cluster.local:9000/data/](https://hdfs-namenode.default.svc.cluster.local:9000/data/).

Los Angeles Crime Data^{1 2} : This dataset comes from the public data repository of the United States Government³ . It contains crime incident records for the city of Los Angeles from 2010 to 2025, split into two parts (2010–2019 and 2020–2025). The linked pages provide descriptions of the data fields for each column.

Census Blocks (Los Angeles County)⁴ : Dataset containing 2020 census block data for Los Angeles County in [geojson](#) format. It is accompanied by a file containing descriptions of its fields (Census Blocks Fields).

Median Household Income by Zip Code (Los Angeles County)⁵ : Dataset containing data on median household income for each ZIP Code in Los Angeles County. For convenience, the data have been collected and stored in `csv` format, using the character “;” as the delimiter. It refers to 2021 statistics.

LA Police Stations⁶ : Small dataset containing location data for the 21 police stations in the city of Los Angeles.

Queries

Query 1

Rank the parts of the day in descending order according to the percentage of crime records, over the total number of records, that took place on the street (“STREET”). Consider the following parts of the day:

- Morning: 5:00 a.m. – 11:59 a.m.
- Evening: 5:00 p.m. – 8:59 p.m.
- Afternoon: 12:00 p.m. – 4:59 p.m.
- Night: 9:00 p.m. – 4:59 a.m.

Query 2

For each year, find the 3 months with the highest number of recorded crimes. For each year, print those months, the total number of incidents, and the ranking position of each month within that year. The results should be ordered in ascending order by year and descending order by the number of records (see the example in Table 2).

¹<https://data.lacity.org/Public-Safety/Crime-Data-from-2010-to-2019/63jg-8b9z>

²<https://data.lacity.org/Public-Safety/Crime-Data-from-2020-to-Present/2nrs-mtv8>

³<https://catalog.data.gov/dataset>

⁴<https://data.lacounty.gov/maps/lacounty::2020-census-blocks>

⁵http://www.laalmanac.com/employment/em12c_2021.php

⁶<https://geohub.lacity.org/datasets/lahub::lapd-police-stations/explore>

year	month	crime_total	ranking
2010	2	2145	1
2010	3	1492	2
2010	5	54	3
2011	12	4632	1
2011	6	2312	2
2011	4	312	3
⋮			

Table 2: Sample output for Query 2

Query 3

Using the 2020 census population data and the 2021 household income data as reference, compute the average annual per capita income for the 2020–2021 period for each ZIP Code in Los Angeles.

Query 4

For each police station, compute the number of crimes that took place closer to that station than to any other station, as well as the average distance between the station and the locations where those incidents occurred. Display the results sorted in descending order by number of incidents (see the example in Table 3).

division	average_distance	#
77TH STREET	2.208	7045
RAMPART	2.009	4595
FOOTHILL	3.597	3047
PACIFIC	2.739	2132

Table 3: Sample output for Query 4.

Requirements

1. The data are stored mainly in csv format. Is this the most suitable format for processing with Spark? What other alternative formats do you know that can be used, and how do they differ? Choose one of them, convert the data to that format, and compare the performance difference (execution time) against csv for one of the **Queries**.
2. Implement **Query 1** using the DataFrame (with and without UDF) and RDD APIs. Run both implementations with 2 executors (1 core, 2GB memory). Is there a difference in performance among the three implementations? Comment on your findings. (%)
3. Implement **Query 2** using the DataFrame and SQL APIs. Run both implementations with 4 executors (1 core, 2GB memory). Compare and comment on the execution times of the two implementations. (%)
4. Implement **Query 3** using the DataFrame and RDD APIs. Run both implementations with 3 executors (1 core, 2GB memory). Compare and comment on the execution times for each implementation. (20%)

5. Implement **Query 4** using the DataFrame or SQL API. For the joins that are performed, report which strategies are selected by the Spark optimizer and comment on those choices. In order to study vertical and horizontal scalability, you are asked to run your implementation as follows:
- A. On 2 executors with the following configurations:
- 1 core, 2 GB memory
 - 2 cores, 4GB memory
 - 4 cores, 8GB memory
- B. Using total resources of 8 cores and 16GB of memory with the following configurations:
- 2 executors $\times$ 4 cores, 8GB memory
 - 4 executors $\times$ 2 cores, 4GB memory
 - 8 executors $\times$ 1 core, 2GB memory
- Comment on the results. (20%)
6. For **Query 3** (DataFrame API) and **Query 4**, use the methods [hint](#) and [explain](#) to determine which join strategies are used by the Catalyst optimizer. Experiment by prompting Spark to use different strategies among BROADCAST, MERGE, SHUFFLE_HASH, and SHUFFLE_REPLICATE_NL, and comment on the impact on performance. Which of Spark's available join strategies is/are the most appropriate, and why?

Deliverables - Submission Terms

- The project must be completed strictly on an individual basis.
- The submission deadline will be announced on [helios](#) via a link that will open soon.
- The project accounts for 40% of the overall course grade. For a grade to be recorded, each group must submit a report **and** successfully pass the mandatory oral examination on the project topic. The examination will take place after project submission (a relevant schedule will be posted on helios).
- The deliverable must be submitted as a .pdf file named after the student registration numbers of the group members, separated by underscores (or the registration number of the student in the case of a single-member group), e.g. 03100000.zip, or 03100000_03100001.zip (depending on the number of members in the group). The file will contain a report (strictly limited to what is requested in the assignment) that includes the answers to the requirements, as well as a mandatory link to a repository (github, gitlab, bitbucket, etc.) containing the code you implemented, along with any scripts/how-tos needed to run it. In addition, a file declaring LLM usage must be included (details below). All submissions are strictly subject to the academic ethics code of NTUA and the School of Electrical and Computer Engineering.

Your code must not change from the day the report is submitted until the grading of the course is completed. If this happens, your grade will be ZERO (0).

- The use of LLM tools is permitted. Undeclared or misleading use is not permitted. In the file LLM_USAGE.md, declare honestly in a table, for each section of the project (code/report text), the following: a) the tool that was used, b) the extent of its use (e.g. "it generated the initial skeleton"), and c) which part was completed by you without using an LLM.
- Each student may implement their code in Scala, Java, or Python. It is mandatory to use resources from the laboratory infrastructure made available to you for the needs of the project. In addition, the Kubernetes infrastructure must contain evidence of the latest

execution of each of your implementations. In all cases, the examination will require a live demonstration of your code.

- Questions and clarifications about the project will be handled through the forum on the course page at [helios](#). Do not send questions to the instructors' or teaching assistants' email addresses.